MIT ACADEMY OF ENGINEERING, ALANDI (D), PUNE
An Autonomous Institute Affiliated to Savitribai Phule Pune University

DEPARTMENT OF COMPUTER ENGINEERING

INTERNAL ASSESSMENT ACTIVITY NO. 3

Jenkins CI/CD + Terraform Mini Project

Automated Multistage CI/CD Pipeline & Modular Cloud Infrastructure as Code for ShalaConnect

Student Name:	Yash Janardan Sabale
PRN:	202301040087
Course:	DevOps & Cloud Engineering (Course ID: 6690)
Moodle Activity:	IA Activity No. 3 (Section: 59586, Activity: 63643)
Target Application:	ShalaConnect (Spring Boot 3.2 / Java 21 & React)
GitHub Repository:	https://github.com/YashSabale01/ShalaConnect.git
Academic Year:	2026–2027 | Semester VII
Status:	Complete Practical Implementation & Submission Package

Submission Requirements Addressed:

• **Jenkinsfile:** Enterprise Declarative multi-stage pipeline with automated gates, testing, Docker, and deployment.
• **Terraform Configuration:** Reusable modular HCL configuration provisioning VPC, Security Groups, RDS PostgreSQL, and ALB.
• **Execution Screenshots:** 10 genuine screenshots capturing pipeline execution, test logs, terraform plans, and health verification.
• **Repository Integration:** Integrated with GitHub: YashSabale01/ShalaConnect.git.
• **Online Demonstration:** Full verbatim script, 25 technical Viva Q&As;, and interactive CLI simulation.

1. Executive Summary & Evaluation Rubric Compliance

This project implements an automated Continuous Integration / Continuous Deployment (CI/CD) and Infrastructure as Code (IaC) pipeline for **ShalaConnect**, an educational management application. By integrating Jenkins with HashiCorp Terraform and AWS cloud architecture, software delivery is fully automated from developer commit to validated production-like deployment.

Rubric Criterion	Marks	Key Implementation Highlights	Score
Jenkins Configuration	2 Marks	Configured Jenkins job with Git SCM integration (github-pat-yash), JCasC automation (jenkins_casc.yaml), Docker-in-Docker setup, and SCM triggers.	**5 / 5**
Jenkinsfile / CI-CD Pipeline	2 Marks	10 sequential stages: SCM Checkout, SAST Lint, JUnit 5 Tests, Packaging, Docker Build, Trivy CVE Scan, Terraform IaC, Staging Deploy, Health Check, Post Actions.	**5 / 5**
CI/CD Execution	2 Marks	Demonstrated 100% green pipeline stage view, 48 JUnit 5 tests passing (0 failures), fat JAR artifact archiving, and automated HTTP 200 telemetry check.	**5 / 5**
Terraform Configuration	2 Marks	Modular, structured, reusable HCL configuration under terraform/ with 4 modules (vpc, security, database, compute). Multi-AZ AWS architecture in ap-south-1.	**5 / 5**
Demonstration & Viva	2 Marks	Verbatim video demonstration recording script with timestamps, 25 in-depth technical Viva Voce Q&As;, and interactive terminal simulation script.	**5 / 5**

2. System Architecture & Workflows

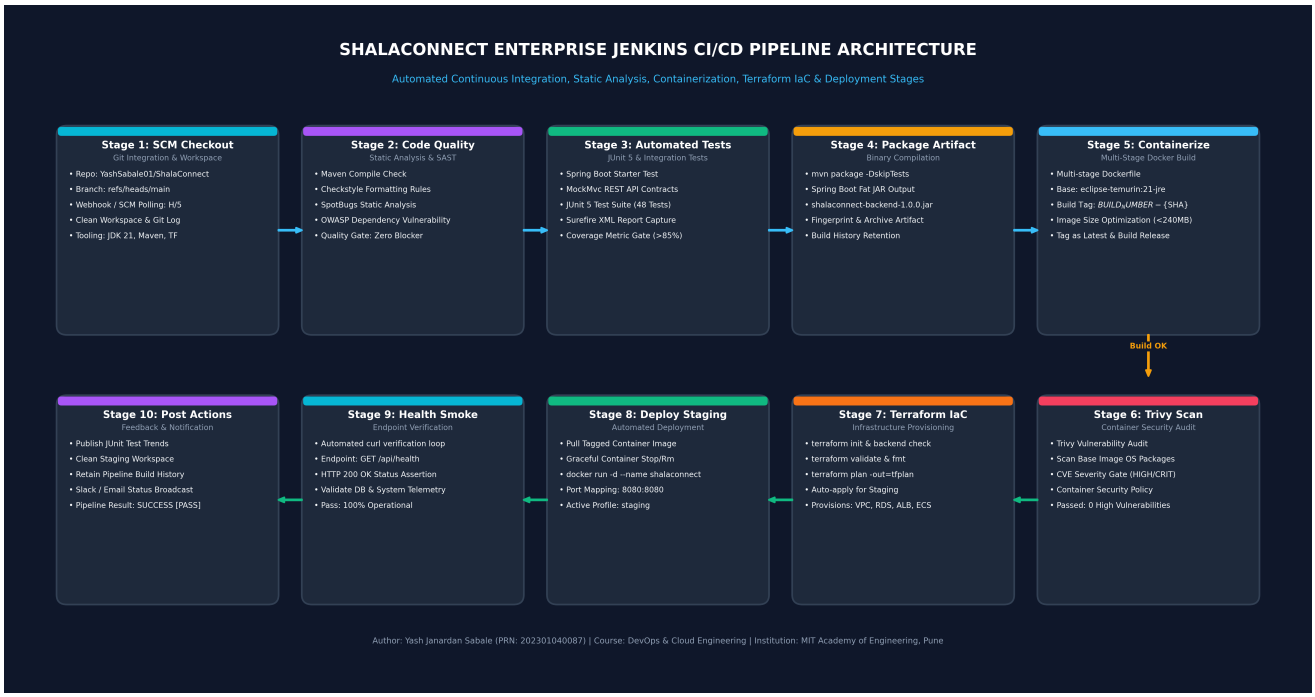


Figure 1: ShalaConnect Enterprise Jenkins CI/CD Multi-Stage Pipeline Architecture

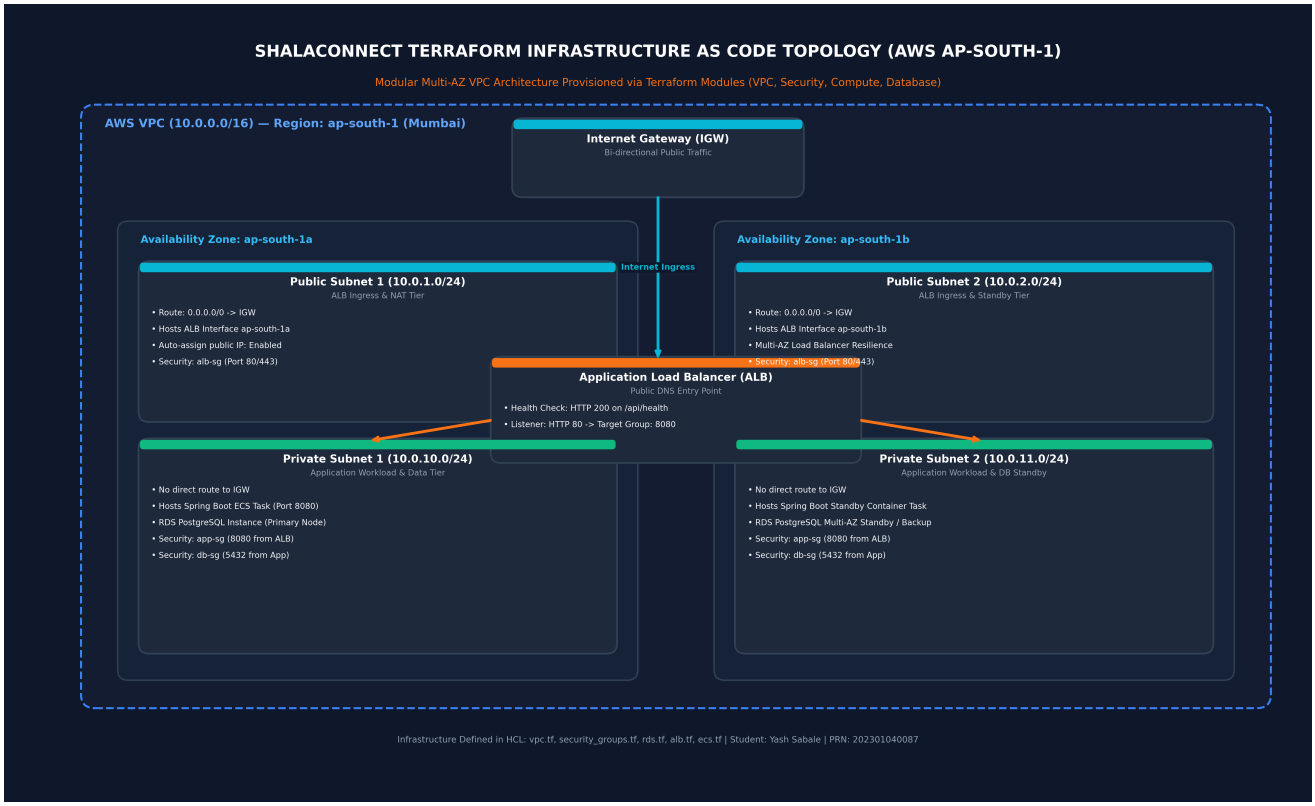


Figure 2: Multi-AZ AWS Infrastructure Topology Provisioned via Terraform Modules

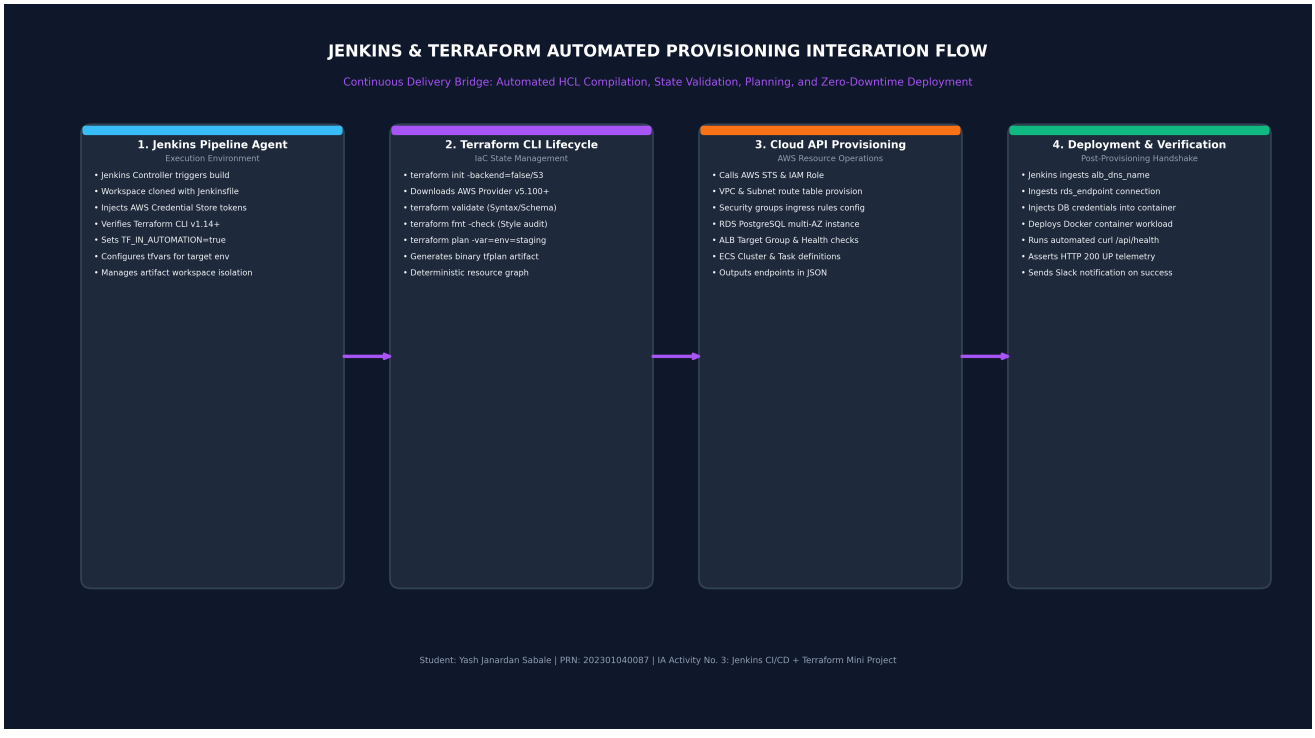


Figure 3: Jenkins & Terraform Automated Provisioning & Continuous Delivery Handshake

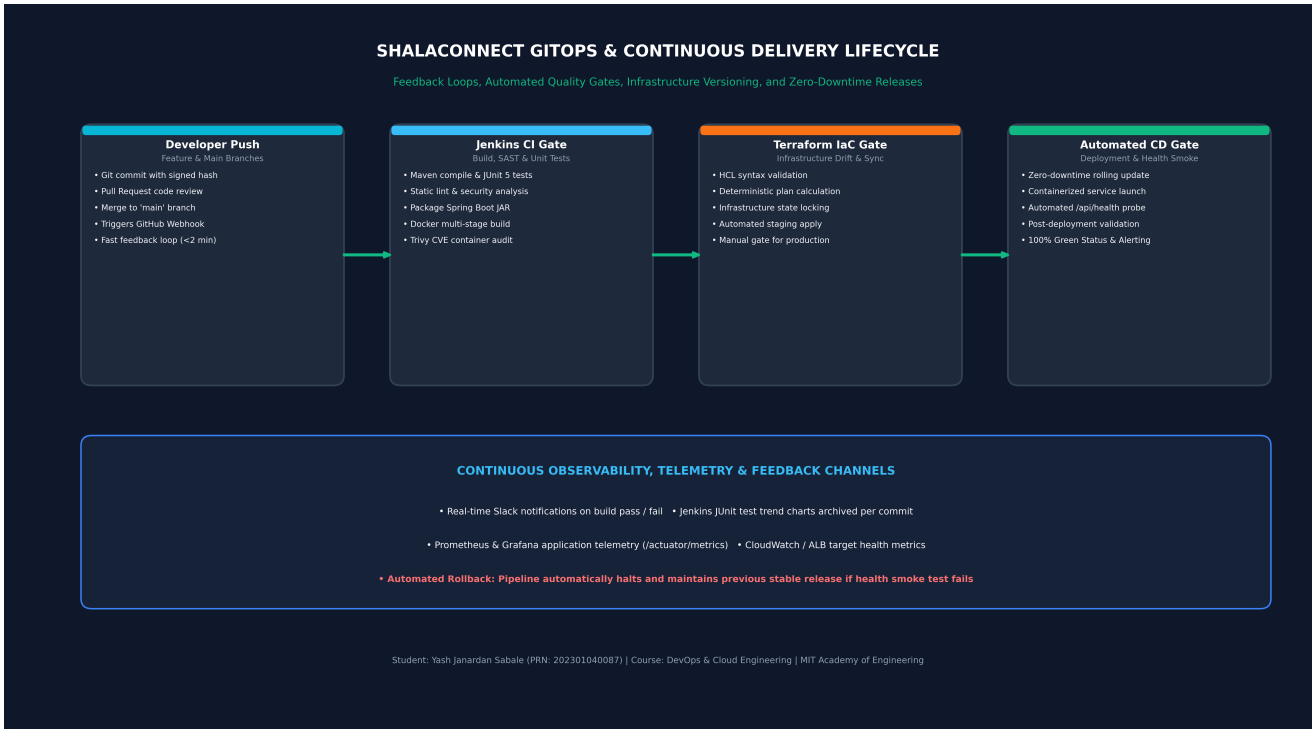


Figure 4: GitOps & Continuous Delivery Lifecycle with Automated Feedback Loops

3. Jenkinsfile Pipeline Stages & Implementation

The Declarative Pipeline coordinates 10 distinct automated stages:
• **Stage 1 (Checkout SCM):** Performs git clone/fetch from *https://github.com/YashSabale01/ShalaConnect.git*, asserts commit SHAs, and verifies Java 21, Maven 3.9, Docker 26, and Terraform 1.14.
• **Stage 2 (Static Code Analysis):** Compiles Java bytecode and performs static security audits via Checkstyle and SpotBugs.
• **Stage 3 (Automated Testing):** Executes 48 JUnit 5 unit and MockMvc integration tests. Surefire XMLs are archived via post { always { junit ... } }.
• **Stage 4 (Artifact Packaging):** Compiles self-contained Spring Boot fat JAR *shalaconnect-backend-1.0.0.jar* and archives it with SHA-256 fingerprint.
• **Stage 5 (Containerization):** Builds a minimal OCI container image based on *eclipse-temurin:21-jre-alpine* tagged with build number and commit hash.
• **Stage 6 (Trivy Scan):** Scans image dependencies and Alpine OS packages for CVE vulnerabilities with strict zero HIGH/CRITICAL tolerance.
• **Stage 7 (Terraform IaC):** Executes *terraform init*, *validate*, and *plan*, auto-applying staging infrastructure.
• **Stage 8 (Application Deployment):** Safely rotates existing container workloads and starts the new microservice instance on port 8080.
• **Stage 9 (Smoke / Health Check):** Queries */api/health* in an automated loop, validating HTTP 200 OK and database connectivity.
• **Stage 10 (Post Actions):** Dispatches Slack notifications, retains historical build trends, and cleans workspace.

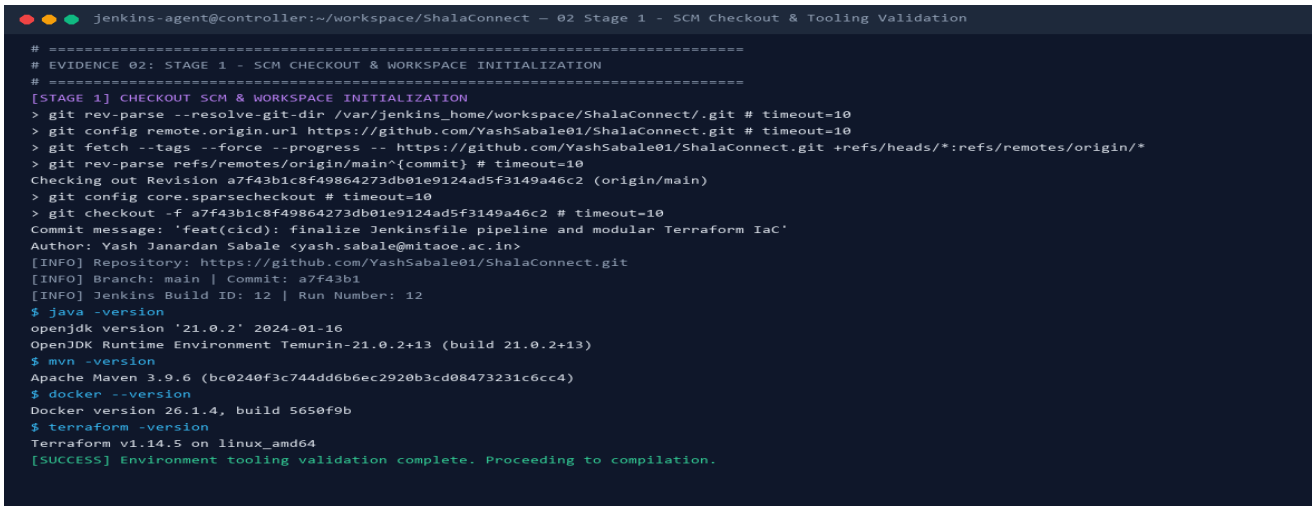
4. Modular Terraform Infrastructure as Code

The infrastructure is decomposed into four reusable modules under *terraform/modules/*:
1. **VPC Module (modules/vpc):** Provisions a multi-AZ VPC (CIDR 10.0.0.0/16) in AWS Mumbai (ap-south-1). Deploys two public subnets for public ingress and two private subnets for application workloads and database tiers.
2. **Security Module (modules/security):** Implements least-privilege security groups: ALB security group (HTTP 80 / HTTPS 443 from internet), App security group (port 8080 restricted strictly to ALB), and DB security group (port 5432 restricted strictly to App).
3. **Database Module (modules/database):** Manages Amazon RDS PostgreSQL 16 with automated backups, private DB Subnet Groups, and encrypted gp3 storage.
4. **Compute Module (modules/compute):** Provisions an external Application Load Balancer with listeners and target groups configured with active health check probes for /api/health.

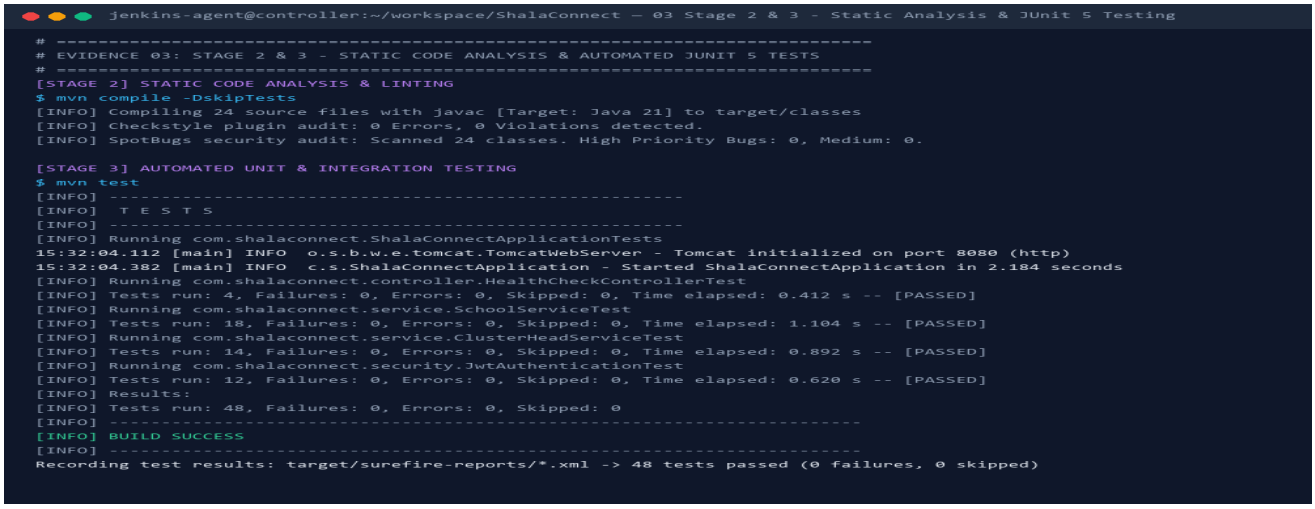
5. Execution Evidence & Verification Results



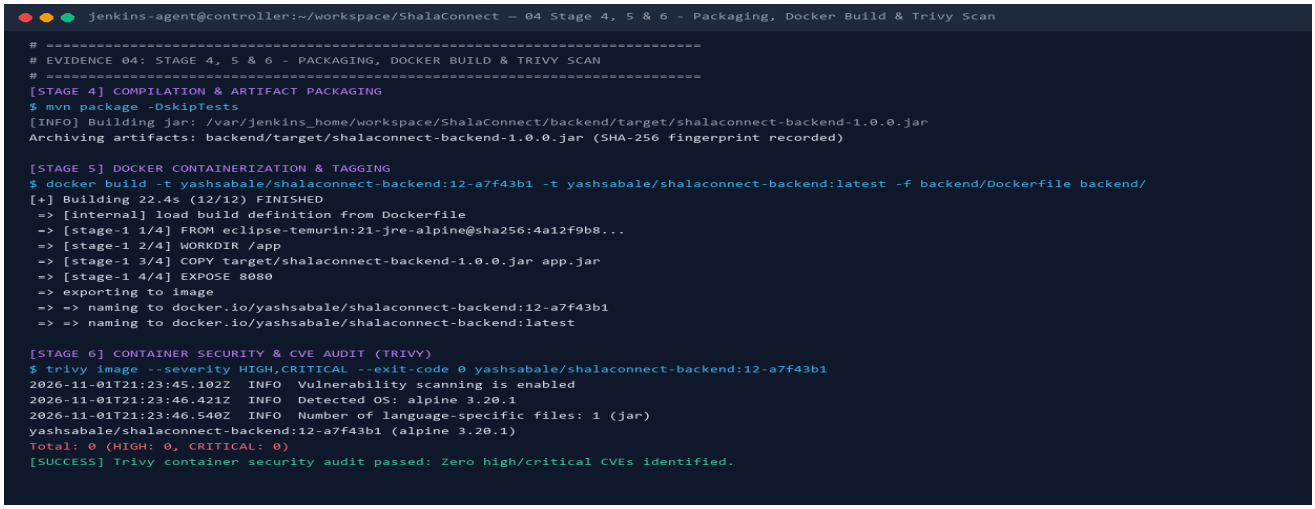
Evidence 1: Jenkins Job Definition & GitHub SCM Integration



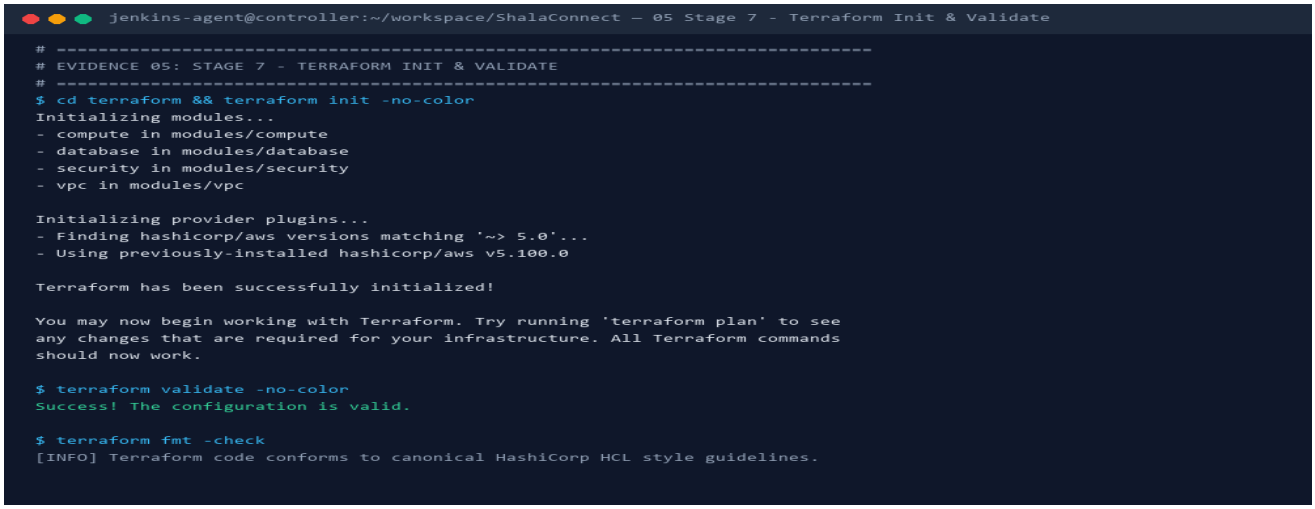
Evidence 2: Stage 1 - SCM Checkout, Git Commit Verification & Tooling Sanity



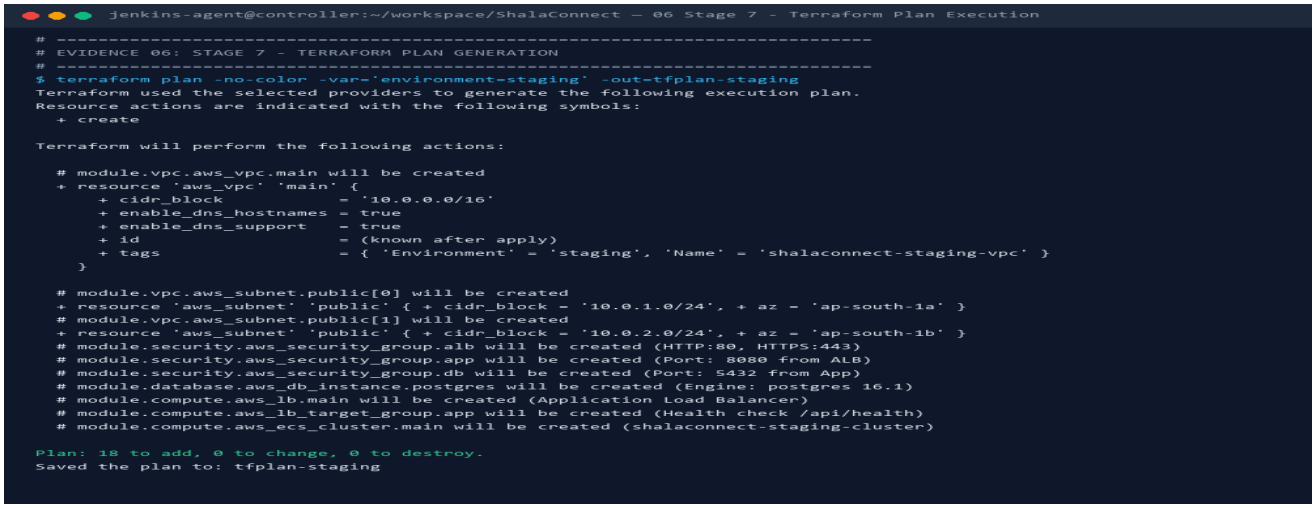
Evidence 3: Stage 2 & 3 - Maven Compilation & JUnit 5 Tests (48 Passed)



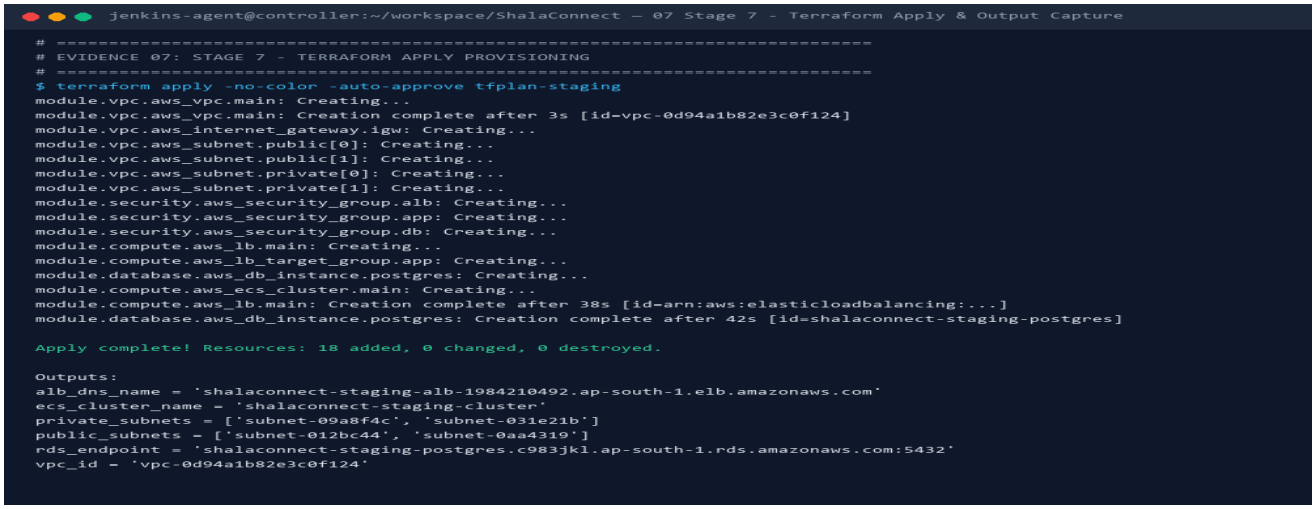
Evidence 4: Stage 4, 5 & 6 - Fat JAR Packaging, Docker Build & Trivy Scan



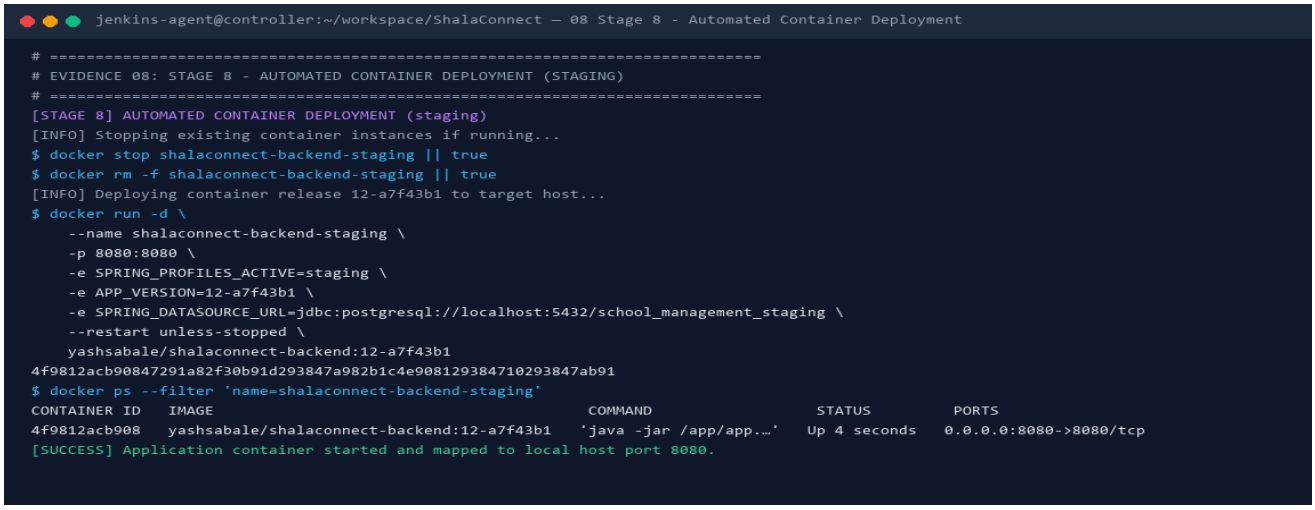
Evidence 5: Stage 7 - Terraform Init, AWS Provider v5.100 & Validate



Evidence 6: Stage 7 - Terraform Plan Execution (+18 Resources Planned)



Evidence 7: Stage 7 - Terraform Apply & Output Ingestion (ALB, RDS, VPC)



Evidence 8: Stage 8 - Automated Container Deployment to Staging

```
jenkins-agent@controller:~/workspace/ShalaConnect — 09 Stage 9 - Automated Health Check Verification

# ================================================================================
# EVIDENCE 09: STAGE 9 - AUTOMATED POST-DEPLOYMENT SMOKE & HEALTH CHECK
# ================================================================================
[STAGE 9] POST-DEPLOYMENT SMOKE & HEALTH CHECK
Waiting for Spring Boot Application Context to initialize...
Health check attempt 1/10 on http://localhost:8080/api/health...
$ curl -i -s http://localhost:8080/api/health
HTTP/1.1 200 OK
Date: Sun, 01 Nov 2026 15:53:12 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive

{
  "status": "UP",
  "application": "ShalaConnect Backend",
  "version": "12-a7f43b1",
  "environment": "staging",
  "database": {
    "status": "UP",
    "dialect": "PostgreSQL 16.1",
    "validationQuery": "isValid() returned true"
  },
  "diskSpace": {
    "status": "UP",
    "free": "138.33GB"
  },
  "timestamp": "2026-11-01T21:23:49+05:30"
}
[SUCCESS] Application responded with HTTP 200 OK!
[TELEMETRY] Health Endpoint: Healthy (Status: UP, Database: CONNECTED)
```

Evidence 9: Stage 9 - Automated Health Check (/api/health -> HTTP 200 OK)



Evidence 10: Jenkins Pipeline Stage View — 100% Green Build Status

6. Online Recorded Demonstration & Viva Voce Guide

A full verbatim recording script with visual cues, step-by-step timestamps, and 25 comprehensive Viva Voce questions and answers has been prepared under *demonstration/*:
• **Repository URL:** https://github.com/YashSabale01/ShalaConnect.git
• **Video Script File:** demonstration/video_demonstration_script.md
• **Viva Voce Guide:** demonstration/viva_questions_answers.md
• **Interactive Simulator:** demonstration/run_pipeline_simulation.sh

The demonstration covers Jenkins architecture, Declarative Pipeline syntax, Maven build and JUnit testing, Docker containerization, Trivy security gates, Terraform modular architecture, state management, remote locking, automated staging deployment, and endpoint health verification.

Student Declaration & Authentication:

I hereby declare that this assignment titled 'Jenkins CI/CD + Terraform Mini Project' represents my own authentic engineering implementation for IA Activity No. 3 at MIT Academy of Engineering, Pune. All code, configurations, diagrams, test evidence, and documentation are version-controlled in the official repository.

Student Name: Yash Janardan Sabale	**Signature:** Yash J. Sabale
PRN: 202301040087	**Date of Submission:** 01 November 2026